1. PROJECT IMPLEMENTATION

Project Title

RAG-Based Customer Support Assistant using LangGraph & HITL

Implementation Steps

1. Document Ingestion

- Load PDF using PyPDFLoader
- Extract text content

2. Chunking

- Split document into smaller chunks (500–1000 tokens)
- Use overlap (50–100 tokens) to maintain context

3. Embedding Generation

- Use embedding model (e.g., OpenAI / HuggingFace)
- Convert chunks → vector format

4. Vector Storage

- Store embeddings in **ChromaDB**
- Enable fast similarity search

5. Query Processing

- User enters query
- Query converted into embedding
- Similar chunks retrieved

6. LLM Processing

- Retrieved chunks + query → sent to LLM
- LLM generates contextual answer

7. LangGraph Workflow

Flow:

Input → Processing Node → Decision → Output Node / HITL

8. Conditional Routing

- If answer confidence LOW → HITL
- If context missing → HITL
- Else → Return response

9. HITL (Human-in-the-loop)

- Escalate to human agent
- Human response stored & returned

2. HIGH-LEVEL DESIGN (HLD)

1. System Overview

- **Problem:** Automated customer support with accurate answers
- **Solution:** RAG system with intelligent routing & escalation

2. Architecture Components

User Interface (Web/CLI)
↓
Document Ingestion Pipeline
↓
Chunking Module
↓
Embedding Model
↓
ChromaDB (Vector DB)
↓
Retriever
↓
LLM Layer
↓
LangGraph Workflow Engine
↓
Routing Layer
↓
Output / HITL

3. Component Description

- **Document Loader:** Loads PDF
- **Chunking:** Splits text
- **Embedding Model:** Converts text → vectors
- **Vector Store:** Stores embeddings
- **Retriever:** Finds relevant chunks
- **LLM:** Generates answer
- **LangGraph:** Controls workflow
- **Routing Layer:** Decision making
- **HITL Module:** Human escalation

4. Data Flow

PDF → Chunk → Embedding → Store → Query → Retrieve → LLM → Response

5. Technology Choices

- **ChromaDB:** Fast & lightweight vector DB
- **LangGraph:** Graph-based workflow control
- **LLM:** GPT / Open-source model

6. Scalability

- Use batching for large documents
- Index optimization for fast retrieval
- Cache frequent queries

3. LOW-LEVEL DESIGN (LLD)

1. Modules

Document Processing Module

- Input: PDF
- Output: Raw text

Chunking Module

```
chunks = split_text(text, size=500, overlap=100)
```

Embedding Module

```
embedding = model.encode(chunk)
```

Vector Storage

```
db.add(chunks, embeddings)
```

Retrieval Module

```
results = db.similarity_search(query)
```

Query Processing

- Combine query + retrieved chunks

Graph Execution

- Executes workflow nodes

HITL Module

- Handles escalation

2. Data Structures

Document

```
{
  "id": "doc1",
  "content": "text"
}
```

Chunk

```
{
  "chunk_id": 1,
  "text": "..."
}
```

Embedding

[0.23, 0.67, 0.12]

Query-Response

```
{
  "query": "...",
  "response": "...",
  "confidence": 0.85
}
```

Graph State

```
{
  "query": "",
  "chunks": [],
  "response": "",
  "status": ""
}
```

3. Workflow Design (LangGraph)

Nodes:

- Input Node
- Processing Node
- Output Node

Edges:

Input → Processing → Decision → Output / HITL

4. Conditional Routing

- Confidence < threshold → HITL
- No chunks → HITL

- Complex query → HITL

5. API Design

Input

```
{
  "query": "What is refund policy?"
}
```

Output

```
{
  "answer": "...",
  "source": "...",
  "confidence": 0.92
}
```

6. Error Handling

- No data → return fallback
- LLM failure → retry
- No match → escalate

4. TECHNICAL DOCUMENTATION

1. Introduction

RAG (Retrieval-Augmented Generation) combines:

- Retrieval (search)
- Generation (LLM)

Why needed?

- Avoid hallucination
- Provide accurate answers
- Use real documents

2. System Architecture Explanation

- Data flows from ingestion → retrieval → generation
- LangGraph controls logic
- Routing decides output

3. Design Decisions

- Chunk size: 500–1000 (balance context + speed)
- Embedding: semantic similarity

- Retrieval: top-k search
- Prompt: context + query

4. Workflow Explanation

- LangGraph manages node transitions
- State carries query & response

5. Conditional Logic

- Intent detection
- Confidence-based routing

6. HITL Implementation

- Triggered when:
    - Low confidence
    - Complex queries
- Human improves accuracy

7. Challenges & Trade-offs

- Accuracy vs speed
- Cost vs performance
- Chunk size vs quality

8. Testing Strategy

- Unit testing modules
- Test queries:
    - “What is refund policy?”
    - “How to contact support?”

9. Future Enhancements

- Multi-document support
- Chat memory
- Feedback learning
- Deployment (cloud)